Level 2

HACKATHON PROJECT SUBMISSION

Fundamentals of Web Development

STUDENT ATTENDANCE TRACKER

Student Details	
Student Name	Asha .A
Register Number	25132010500122013
Department	BCA
ID	anm20125132010500122013
College	ARIGNAR ANNA GOVERNMENT ARTS COLLEGE, VILLUPURAM 605602
Date of Submission	27/10/2026
Git-Hub Link	Not provided

1. Problem Statement

Maintaining student attendance manually in notebooks or spreadsheets can take time and may lead to missing entries, duplicate records, and difficulty finding a particular student's attendance. A simple web application can make this process easier by providing a single place to record, review, and search attendance.

The Student Attendance Tracker is a browser-based application developed using HTML, CSS, and JavaScript. It lets a user enter student details, select a date and attendance status, save the record, and view the saved records. The project demonstrates a practical use of basic web-development concepts.

Need for the application

- Reduce the effort involved in organizing daily attendance.
- Keep student attendance records in a consistent format.
- Make it easier to search records by name, register number, department, or status.
- Show a quick summary of total, present, and absent records.

2. Objectives

The main objectives of the Student Attendance Tracker are:

1. Create a clear and user-friendly dashboard for attendance management.
2. Provide a form to enter the student name, register number, department, date, status, session, and optional notes.
3. Validate required entries before saving attendance.
4. Prevent duplicate entries for the same register number, date, and session.
5. Store records in browser local storage so that they remain available in the same browser.
6. Display saved attendance records as cards.
7. Search records using keywords and filter by Present or Absent status.
8. Display dashboard counts for total records, present records, and absent records.

Expected outcome

The completed application provides a working front end for recording and reviewing attendance. It is suitable as a small educational prototype. It does not use a server or shared database, so the records are stored only in the browser where the application is opened.

3. Architecture

The application is a client-side web application. HTML defines the sections and form fields, CSS controls the layout and responsive appearance, and JavaScript handles navigation, form submission, record storage, duplicate checks, search, filtering, and dashboard totals.

Application structure

• index.html — page structure, navigation buttons, attendance form, dashboard, records area, and search controls.
• style.css — colours, spacing, cards, form controls, navigation, and mobile layout.
• script.js — local storage, record creation, validation, rendering, searching, filtering, and summary counts.

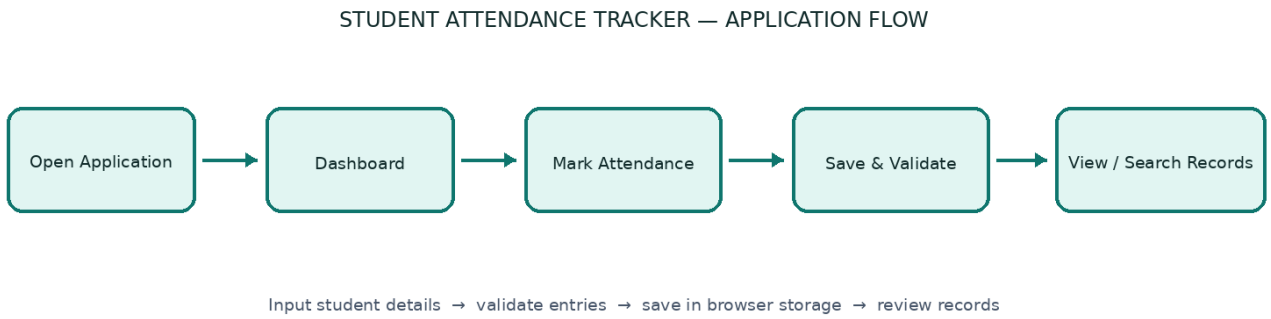


Figure 1. Application flow from opening the dashboard to reviewing attendance records.

Technologies used

HTML: Structures the application and collects user input.
CSS: Styles the page, cards, buttons, form fields, and responsive layout.
JavaScript: Implements application logic and updates the page without reloading.
Browser local storage: Stores records in the current browser for later viewing.

4. Practical Implementation

The application was implemented as three linked files. The HTML page contains the dashboard, navigation, attendance form, record display area, and search section. The CSS file gives the interface a consistent colour scheme and adjusts the layout for smaller screens. The JavaScript file loads records from local storage, adds new records, checks for duplicate entries, updates summary counts, and renders search results.

The form asks for a student name, register number, department, date, attendance status, session, and optional notes. When the form is submitted, JavaScript checks the required values and verifies that a record with the same register number, date, and session does not already exist. A valid entry is added to the record list and saved in local storage.

4.1 Features Implemented

Dashboard: Displays the total number of records and the Present and Absent counts.
Mark Attendance: Accepts student details and attendance information through a form.
Duplicate check: Prevents the same student from being recorded twice for the same date and session.
Attendance Records: Displays saved records with their status, department, date, and session.
Search and filter: Searches student details and filters records by attendance status.
Browser storage: Uses localStorage to retain records in the current browser.
Responsive layout: Rearranges cards and form fields for smaller screens.

4.2 Source Code

The source code is organized into HTML, CSS, and JavaScript files. The following code is included in the project ZIP file as editable source code. Line numbers are added in this report for easier explanation.

HTML

```
001  <!DOCTYPE html>
002  <html lang="en">
003   <head>
004    <meta charset="utf-8"/>
005    <meta content="width=device-width, initial-scale=1.0" name="viewport"/>
006    <title>
007     Student Attendance Tracker
008    </title>
009    <link href="style.css" rel="stylesheet"/>
010   </head>
011   <body>
012    <header class="hero">
013     <span class="badge">
014      CAMPUS HACKATHON PROJECT
015     </span>
016     <h1>
017      Student Attendance Tracker
018     </h1>
019     <p>
020      Record, review and track student attendance in one simple portal.
021     </p>
022    </header>
023    <nav class="nav">
024     <button onclick="showSection('home')">
025      Dashboard
026     </button>
027     <button onclick="showSection('mark')">
028      Mark Attendance
029     </button>
030     <button onclick="showSection('records')">
031      Attendance Records
032     </button>
033     <button onclick="showSection('search')">
034      Search
035     </button>
036    </nav>
037    <main>
038     <section class="section active" id="home">
039      <div class="welcome">
040       <div>
041        <span class="eyebrow">
042         WELCOME
043        </span>
044        <h2>
045         Student Attendance Dashboard
046        </h2>
047        <p>
048         Record daily attendance, view saved records and check attendance summaries for students.
049        </p>
050        <button class="primary" onclick="showSection('mark')">
051         Mark Attendance
052        </button>
053       </div>
054       <div class="illustration">
055        ■
056        <small>
057         Track it. Review it. Improve it.
058        </small>
059       </div>
060      </div>
061      <div class="stats">
062       <div class="stat">
063        <span>
064         Total Records
065        </span>
```

HTML (continued)

```
        <strong id="totalCount">
         0
        </strong>
       </div>
       <div class="stat">
        <span>
         Present
        </span>
        <strong id="presentCount">
         0
        </strong>
       </div>
       <div class="stat">
        <span>
         Absent
        </span>
        <strong id="absentCount">
         0
        </strong>
       </div>
      </div>
     </section>
     <section class="section" id="mark">
      <div class="section-title">
       <div>
        <span class="eyebrow">
         ATTENDANCE
        </span>
        <h2>
         Mark Student Attendance
        </h2>
       </div>
      </div>
      <form class="form-card" id="attendanceForm">
       <div class="grid">
        <label>
         Student Name
         <input id="studentName" placeholder="e.g. Priya" required=""/>
        </label>
        <label>
         Register Number
         <input id="registerNo" placeholder="e.g. BCA001" required=""/>
        </label>
        <label>
         Department
         <select id="department" required="">
          <option value="">
           Select department
          </option>
          <option>
           BCA
          </option>
          <option>
           B.Sc. Computer Science
          </option>
          <option>
           B.Com
          </option>
          <option>
           BA
          </option>
          <option>
           Other
          </option>
         </select>
```

HTML (continued)

```
      </label>
      <label>
       Date
       <input id="attendanceDate" required="" type="date"/>
      </label>
      <label>
       Status
       <select id="status" required="">
        <option value="Present">
         Present
        </option>
        <option value="Absent">
         Absent
        </option>
       </select>
      </label>
      <label>
       Session
       <select id="session" required="">
        <option>
         Full Day
        </option>
        <option>
         Morning
        </option>
        <option>
         Afternoon
        </option>
       </select>
      </label>
     </div>
     <label>
      Notes (optional)
      <textarea id="notes" placeholder="Add a short note if needed" rows="3"></textarea>
     </label>
     <button class="primary" type="submit">
      Save Attendance
     </button>
    </form>
   </section>
   <section class="section" id="records">
    <div class="section-title">
     <div>
      <span class="eyebrow">
       RECORDS
      </span>
      <h2>
       All Attendance Records
      </h2>
     </div>
     <button class="secondary" onclick="clearDemoData()">
      Clear Demo Data
     </button>
    </div>
    <div class="record-grid" id="recordGrid">
    </div>
   </section>
   <section class="section" id="search">
    <div class="section-title">
     <div>
      <span class="eyebrow">
       SEARCH
      </span>
      <h2>
       Find Attendance Records
```

HTML (continued)

```
        </h2>
       </div>
      </div>
      <div class="search-bar">
       <input id="searchInput" oninput="renderRecords()" placeholder="Search by name, register number or
           department..."/>
       <select id="statusFilter" onchange="renderRecords()">
        <option value="All">
         All Statuses
        </option>
        <option>
         Present
        </option>
        <option>
         Absent
        </option>
       </select>
      </div>
      <div class="record-grid" id="searchGrid">
      </div>
     </section>
    </main>
    <div class="toast" id="toast">
    </div>
    <footer>
     &copy; 2026 Student Attendance Tracker &bull; Built with HTML, CSS &amp; JavaScript
    </footer>
    <script src="script.js">
    </script>
   </body>
  </html>
```

CSS

```
*  {
    box-sizing:border-box;
    margin:0;
    padding:0
}
:root  {
    --primary:#0f766e;
    --dark:#102a2a;
    --muted:#64748b;
    --bg:#f3f8f7;
    --card:#fff;
    --border:#dbe7e5
}
body  {
    font-family:Arial,Helvetica,sans-serif;
    background:var(--bg);
    color:var(--dark);
    line-height:1.6
}
.hero  {
    padding:42px 7%;
    background:linear-gradient(135deg,#134e4a,#0f766e);
    color:#fff
}
.hero h1  {
    font-size:clamp(30px,5vw,52px);
    margin:10px 0
}
.hero p  {
    opacity:.93;
    font-size:18px
}
.badge,.eyebrow  {
    font-size:12px;
    font-weight:700;
    letter-spacing:1.5px
}
.badge  {
    background:rgba(255,255,255,.16);
    padding:7px 11px;
    border-radius:20px
}
.nav  {
    display:flex;
    gap:8px;
    justify-content:center;
    flex-wrap:wrap;
    background:#fff;
    border-bottom:1px solid var(--border);
    padding:10px
}
.nav button,.secondary  {
    border:0;
    background:transparent;
    padding:11px 16px;
    cursor:pointer;
    font-weight:700
}
.nav button:hover,.secondary:hover  {
    background:#dff5f1;
    border-radius:8px
}
main  {
    max-width:1150px;
    margin:auto;
```

CSS (continued)

```
    padding:30px 18px
}
.section  {
    display:none
}
.section.active  {
    display:block
}
.welcome  {
    display:grid;
    grid-template-columns:1.7fr 1fr;
    gap:20px;
    background:var(--card);
    padding:35px;
    border-radius:18px;
    box-shadow:0 10px 30px #102a2a12
}
.welcome h2  {
    font-size:32px;
    margin:7px 0 12px
}
.welcome p  {
    color:var(--muted);
    max-width:650px
}
.illustration  {
    display:grid;
    place-items:center;
    background:#dff5f1;
    border-radius:15px;
    font-size:70px;
    text-align:center;
    min-height:150px
}
.illustration small  {
    font-size:13px;
    color:var(--primary)
}
.primary  {
    background:var(--primary);
    color:#fff;
    border:0;
    padding:12px 18px;
    border-radius:9px;
    cursor:pointer;
    font-weight:700;
    margin-top:18px
}
.primary:hover  {
    opacity:.9
}
.stats  {
    display:grid;
    grid-template-columns:repeat(3,1fr);
    gap:18px;
    margin-top:20px
}
.stat  {
    background:#fff;
    padding:22px;
    border-radius:14px;
    border:1px solid var(--border)
}
.stat span  {
    color:var(--muted)
```

CSS (continued)

```
}
.stat strong  {
    display:block;
    font-size:34px;
    color:var(--primary)
}
.section-title  {
    display:flex;
    justify-content:space-between;
    align-items:center;
    margin-bottom:18px
}
.section-title h2  {
    font-size:28px
}
.eyebrow  {
    color:var(--primary)
}
.form-card  {
    background:#fff;
    border:1px solid var(--border);
    border-radius:16px;
    padding:24px;
    box-shadow:0 8px 25px #102a2a0a
}
.grid  {
    display:grid;
    grid-template-columns:repeat(2,1fr);
    gap:16px;
    margin:18px 0
}
label  {
    font-weight:700;
    font-size:14px;
    display:block
}
input,select,textarea  {
    width:100%;
    margin-top:7px;
    padding:12px;
    border:1px solid #cbd5e1;
    border-radius:8px;
    font:inherit;
    background:#fff
}
.record-grid  {
    display:grid;
    grid-template-columns:repeat(3,1fr);
    gap:18px
}
.record-card  {
    background:#fff;
    border:1px solid var(--border);
    border-radius:15px;
    overflow:hidden;
    padding:18px
}
.record-card h3  {
    margin:8px 0
}
.meta  {
    color:var(--muted);
    font-size:13px;
    margin:3px 0
}
```

CSS (continued)

```
.pill  {
    display:inline-block;
    padding:4px 9px;
    border-radius:20px;
    font-size:12px;
    font-weight:700;
    background:#dcfce7;
    color:#166534
}
.pill.absent  {
    background:#fee2e2;
    color:#991b1b
}
.search-bar  {
    display:grid;
    grid-template-columns:1fr 180px;
    gap:10px;
    margin-bottom:20px
}
.search-bar input,.search-bar select  {
    margin:0
}
.toast  {
    position:fixed;
    right:20px;
    bottom:20px;
    background:#102a2a;
    color:#fff;
    padding:13px 17px;
    border-radius:9px;
    display:none;
    z-index:10
}
footer  {
    text-align:center;
    padding:25px;
    color:var(--muted)
}
@media(max-width:800px)  {
    .welcome  {
        grid-template-columns:1fr
    }
    .record-grid  {
        grid-template-columns:repeat(2,1fr)
    }
}
@media(max-width:600px)  {
    .stats,.grid,.record-grid,.search-bar  {
        grid-template-columns:1fr
    }
    .hero  {
        padding:32px 20px
    }
    .welcome  {
        padding:24px
    }
    .illustration  {
        min-height:170px
    }
    .section-title  {
        align-items:flex-start;
        gap:10px;
        flex-direction:column
    }
}
```

JavaScript

```
001  const KEY = "studentAttendanceRecords";
002  let records = JSON.parse(localStorage.getItem(KEY) || "[]");
003
004  if (!records.length) {
005    records = [
006      {id:1,name:"Sample
           Student",registerNo:"BCA001",department:"BCA",date:"2026-10-20",status:"Present",session:"Full
           Day",notes:"Demo record"},
007      {id:2,name:"Demo
           Student",registerNo:"BCA002",department:"BCA",date:"2026-10-20",status:"Absent",session:"Full
           Day",notes:"Demo record"}
008    ];
009    saveRecords();
010  }
011  function saveRecords(){ localStorage.setItem(KEY, JSON.stringify(records)); }
012  function showSection(id){
013    document.querySelectorAll(".section").forEach(s=>s.classList.remove("active"));
014    document.getElementById(id).classList.add("active");
015    if(id==="records" || id==="search") renderRecords();
016    window.scrollTo({top:0,behavior:"smooth"});
017  }
018  function renderRecords(){
019    const q=(document.getElementById("searchInput")?.value || "").toLowerCase().trim();
020    const status=document.getElementById("statusFilter")?.value || "All";
021    const filtered=records.filter(r=>{
022      const hay=[r.name,r.registerNo,r.department,r.date,r.session,r.notes].join(" ").toLowerCase();
023      return hay.includes(q) && (status==="All" || r.status===status);
024    });
025    const html=filtered.length ? filtered.map(recordCard).join("") :
026      '<div class="form-card"><h3>No matching records found.</h3><p>Try another keyword or
           status.</p></div>';
027    const a=document.getElementById("recordGrid"), b=document.getElementById("searchGrid");
028    if(a)a.innerHTML=html; if(b)b.innerHTML=html;
029    updateDashboard();
030  }
031  function recordCard(r){
032    return `<article class="record-card"><span class="pill
           ${r.status==="Absent"?"absent":""}">${r.status}</span>
033    <h3>${escapeHTML(r.name)}</h3><p class="meta">Register No: ${escapeHTML(r.registerNo)}</p>
034    <p class="meta">Department: ${escapeHTML(r.department)}</p><p class="meta">Date:
           ${escapeHTML(r.date)}</p>
035    <p class="meta">Session: ${escapeHTML(r.session)}</p><p>${escapeHTML(r.notes || "No
           notes")}</p></article>`;
036  }
037  document.getElementById("attendanceForm").addEventListener("submit",e=>{
038    e.preventDefault();
039    const registerNo=document.getElementById("registerNo").value.trim();
040    if(!registerNo){showToast("Enter a register number.");return;}
041    const date=document.getElementById("attendanceDate").value;
042    const name=document.getElementById("studentName").value.trim();
043    const status=document.getElementById("status").value;
044    const session=document.getElementById("session").value;
045    if(records.some(r=>r.registerNo.toLowerCase()===registerNo.toLowerCase() && r.date===date &&
           r.session===session)){
046      showToast("A record already exists for this student, date and session.");return;
047    }
048    records.unshift({id:Date.now(),name,registerNo,department:document.getElementById("department").value,d
           ate,status,session,notes:document.getElementById("notes").value.trim()});
049    saveRecords(); e.target.reset(); document.getElementById("attendanceDate").valueAsDate=new Date();
050    showToast("Attendance saved successfully!"); renderRecords(); showSection("records");
051  });
052  function updateDashboard(){
053    document.getElementById("totalCount").textContent=records.length;
054    document.getElementById("presentCount").textContent=records.filter(r=>r.status==="Present").length;
055    document.getElementById("absentCount").textContent=records.filter(r=>r.status==="Absent").length;
```

JavaScript (continued)

```
056  }
057  function clearDemoData(){
058    if(confirm("Clear all attendance
           records?")){records=[];saveRecords();renderRecords();showToast("Records cleared.");}
059  }
060  function showToast(msg){const t=document.getElementById("toast");t.textContent=msg;t.style.display="block
         ";setTimeout(()=>t.style.display="none",2500);}
061  function escapeHTML(v){return
         String(v).replace(/[&<>"']/g,m=>({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;","'":"&#039;"}[m]));}
062  document.getElementById("attendanceDate").valueAsDate=new Date();
063  renderRecords();
```

4.3 Project Execution

How to run the application

1. Extract the project ZIP file into a folder.
2. Keep index.html, style.css, and script.js in the same folder.
3. Open index.html in a modern web browser such as Chrome, Edge, or Firefox.
4. The dashboard opens and displays the current attendance summary.
5. Select “Mark Attendance”, complete the required fields, and choose “Save Attendance”.
6. Open “Attendance Records” to view saved records, or use “Search” to search and filter records.

Working process

The user enters the attendance details and submits the form. JavaScript checks the register number and checks whether a record already exists for the same student, date, and session. If the entry is valid, the record is added to the records array and saved in localStorage. The dashboard counts and record cards are then refreshed. Search text and status selection are used to filter the displayed records.

Note: localStorage is specific to the browser and device. The application does not synchronize records between different computers or browsers.

5. Output Screenshots

The screenshots below were captured from the rendered Student Attendance Tracker application. They demonstrate the dashboard, attendance form, records view, and search/filter feature.

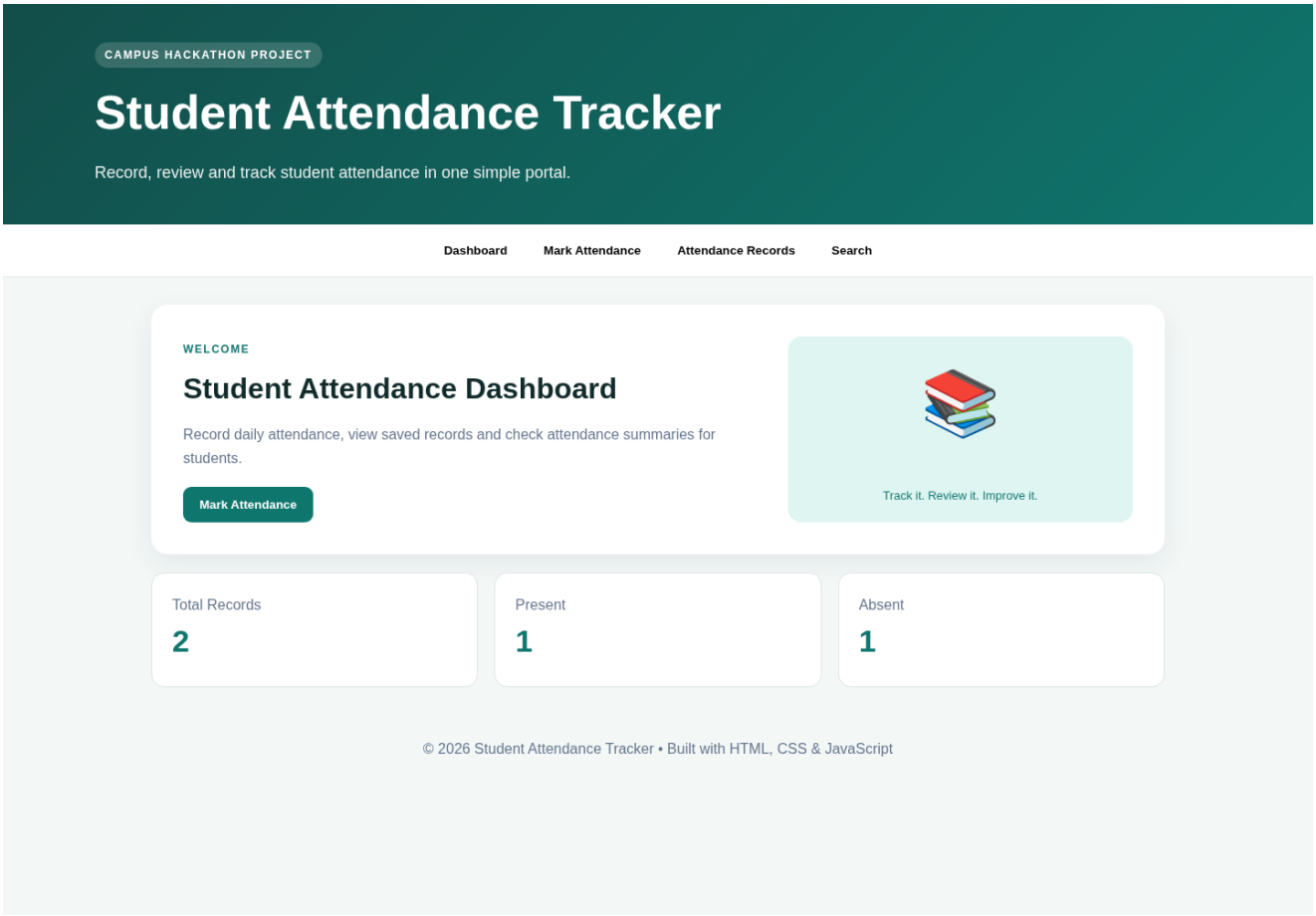


Figure 2. Dashboard with attendance summary.

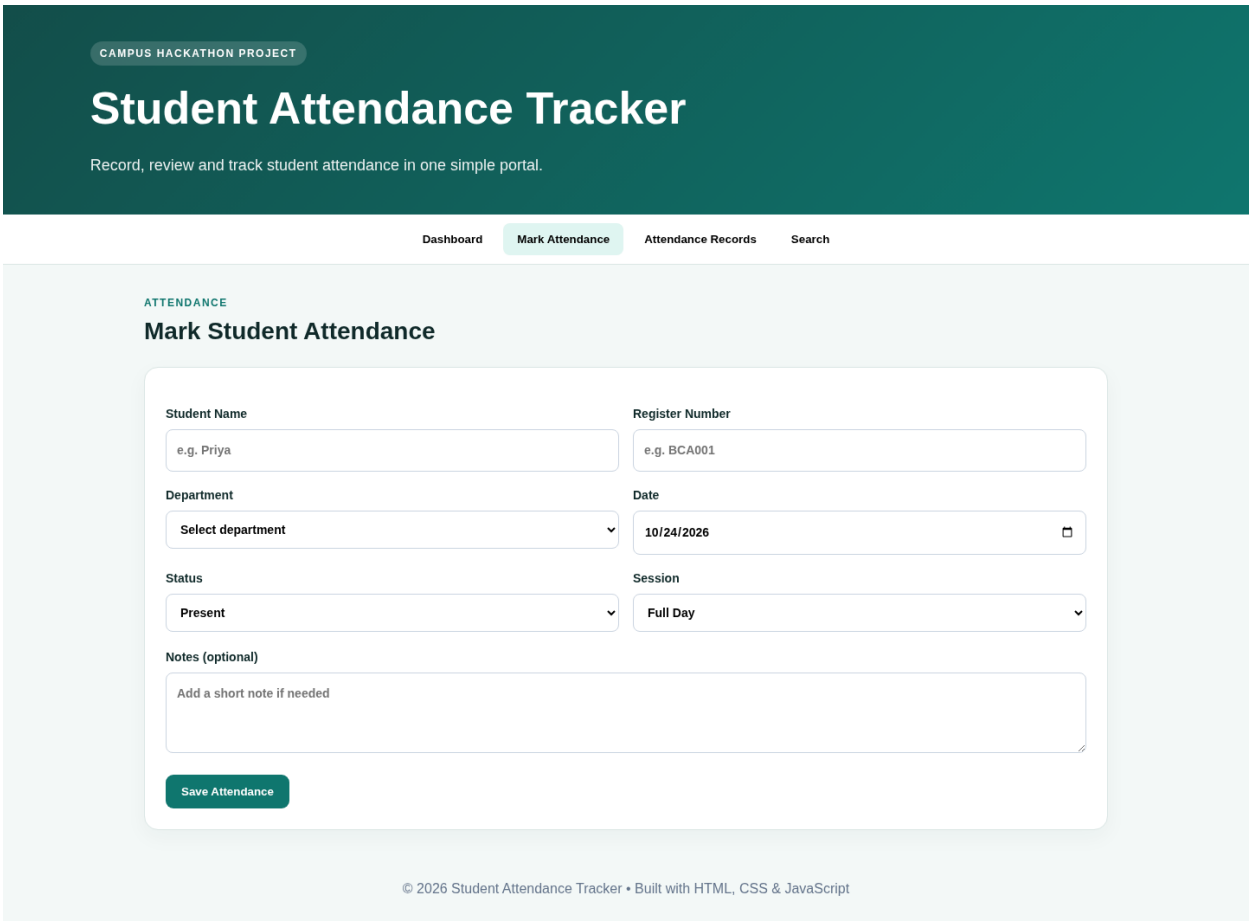


Figure 3. Mark Attendance form.

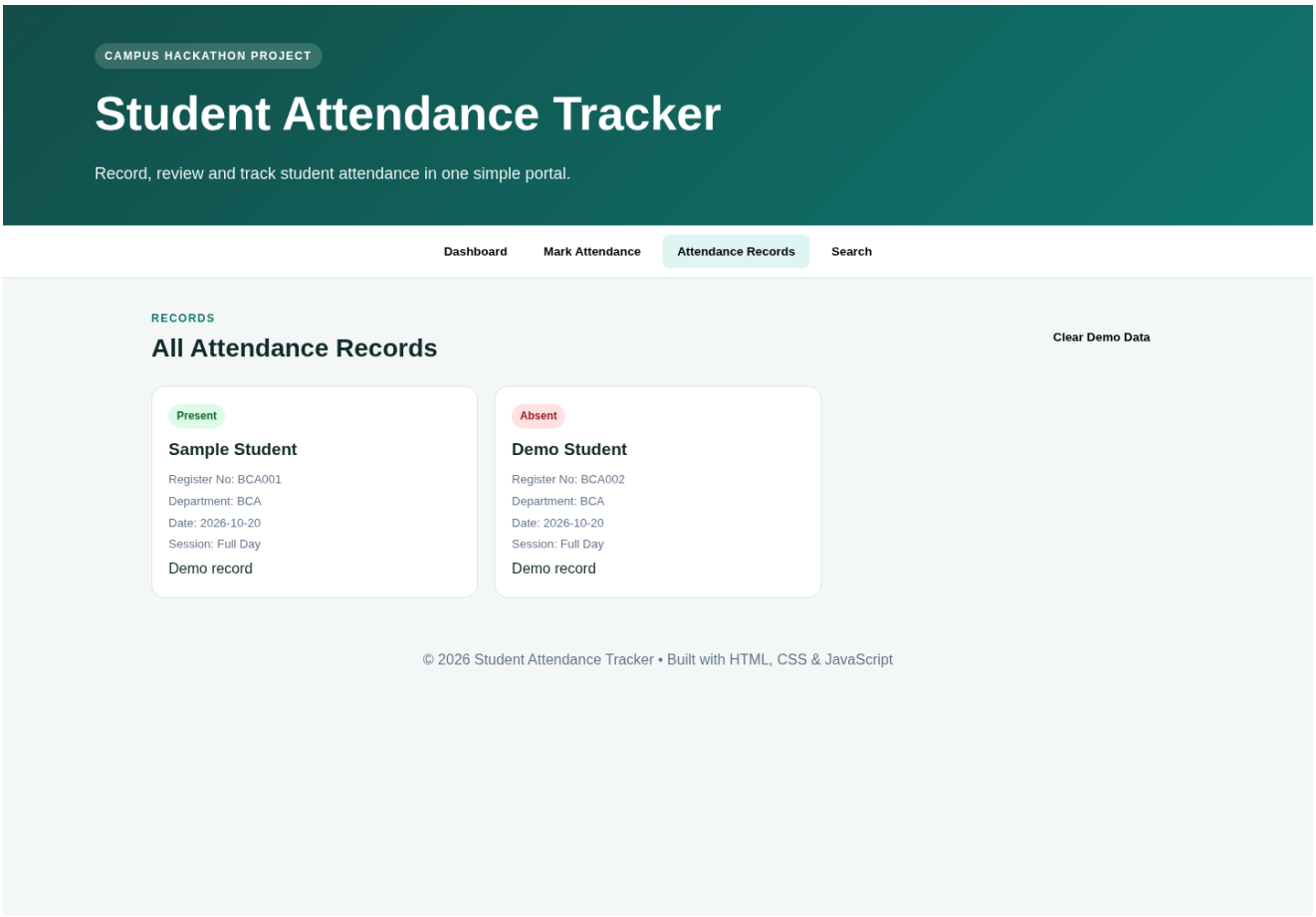


Figure 4. Attendance Records view.

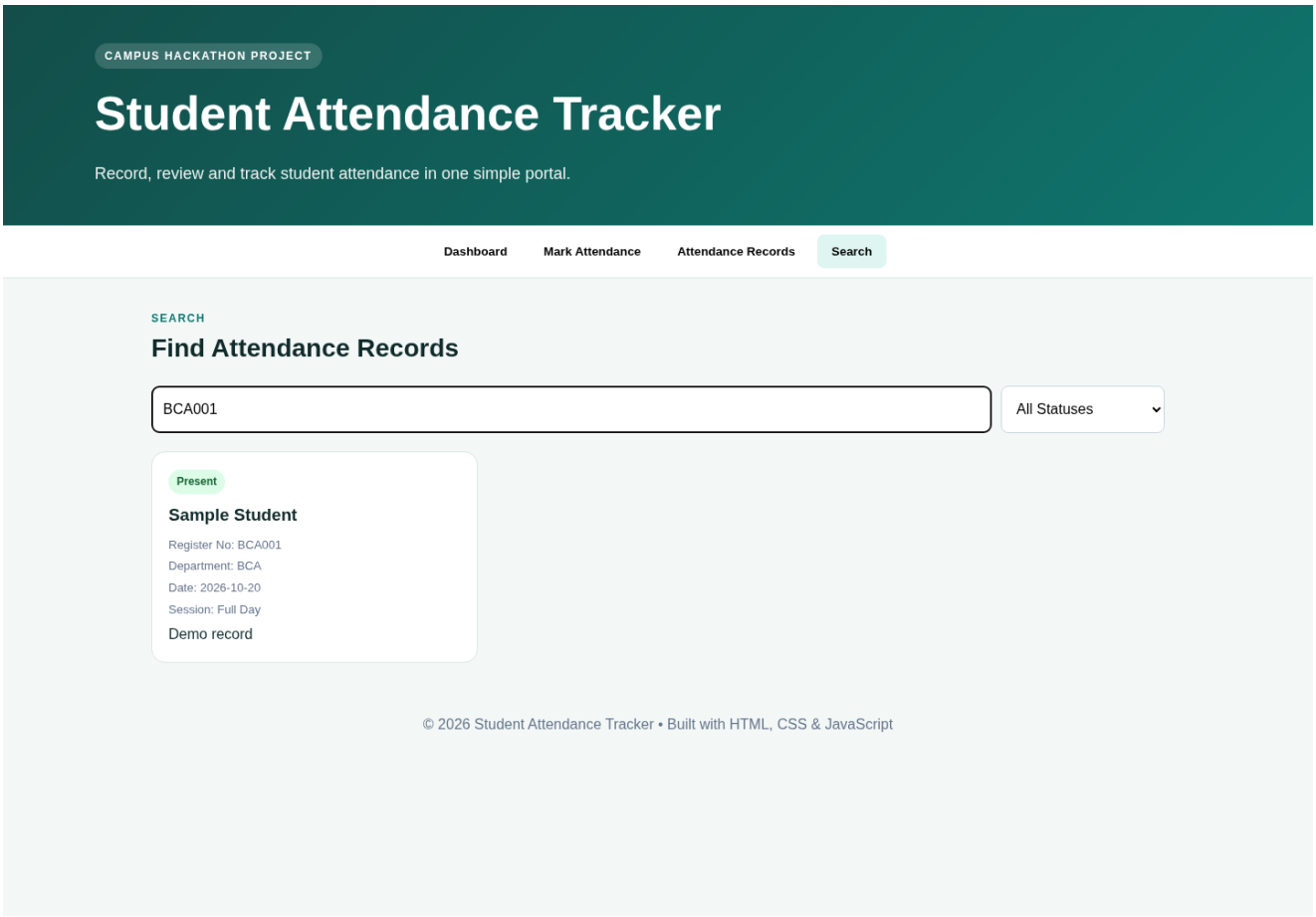


Figure 5. Search results filtered by register number.

6. Technical Explanation

DOM manipulation: JavaScript selects HTML elements using methods such as getElementById and querySelectorAll, then updates text and record-card content.
Event handling: Click handlers switch between sections, while the form submit event saves attendance. The submit handler calls preventDefault to avoid a full page reload.
Arrays and objects: Each attendance record is represented as an object, and all records are stored in an array.
Local storage: JSON.stringify converts the records array to text before saving; JSON.parse restores it when the application opens.
Validation and duplicate detection: Required form fields and a JavaScript check help avoid incomplete or repeated entries.
Search and filtering: The code compares the lower-case search text with record fields and optionally matches the selected attendance status.
HTML escaping: User-entered text is escaped before being inserted into record cards to reduce the risk of interpreting typed text as HTML.

7. Conclusion

The Student Attendance Tracker demonstrates how HTML, CSS, and JavaScript can be combined to create a useful browser-based application. It provides a dashboard, attendance form, record list, duplicate check, search, status filter, and summary counts. The project helped demonstrate form handling, DOM updates, arrays and objects, browser storage, and responsive design.

The application is a prototype for learning purposes. Because it stores information in localStorage, it is best suited to a single browser and does not provide shared access, authentication, or centralized data backup.

8. Future Enhancements

The following improvements could be added in a future version:

1. Connect the application to a server and database so authorized staff can share records.
2. Add login and role-based access for administrators, teachers, and students.
3. Provide attendance percentage reports for each student and class.
4. Add export options for CSV, Excel, or PDF reports.
5. Add date-range filters and department/class-based reports.
6. Add backup and restore functions.
7. Improve accessibility with keyboard navigation and clearer validation messages.
8. Add stronger data validation and privacy protections before using real student information.

— End of Report —

Additional Detailed Explanation

This expanded section supplements the main report with more detailed descriptions of the application workflow, data fields, validation rules, test cases and practical limitations. It follows the same Student Attendance Tracker project and does not introduce a different project.

Detailed workflow

1. Open dashboard: The browser loads the main HTML document, stylesheet and JavaScript file. The dashboard is the initial active section. The summary cards show the number of saved records and the present/absent totals.
2. Enter attendance: The user opens the Mark Attendance section and fills in the student name, register number, department, date, status and session. Notes are optional.
3. Validate: Required form controls check for missing values. JavaScript trims spaces from important text fields and checks that a register number is available.
4. Detect duplicates: The application searches existing records for the same register number, date and session. If a match exists, a warning is shown and the new record is not saved.
5. Save: A valid record is added to the start of the array and the complete array is stored in localStorage as JSON.
6. Display: The record-rendering function creates a card for each entry and updates the summary counters.
7. Search: The user can type a keyword and select a status. The displayed list is narrowed to matching records.

Purpose of each data field

Field	Purpose
Student Name	Identifies the student in the record.
Register Number	Provides a student identifier and is used in duplicate checking.
Department	Groups the student under a department.
Date	Indicates the day attendance was recorded.
Status	Stores Present or Absent.
Session	Identifies Full Day, Morning or Afternoon attendance.
Notes	Optional text for additional context.

Detailed Testing and Expected Results

Testing checks whether the main interactions behave as expected. The tests below can be repeated by opening the project in a browser and using sample data. Use fictional sample student information during demonstration.

No.	Test input/action	Expected result	Reason
1	Open index.html	Dashboard appears	Checks page loading
2	Submit complete new record	Success feedback and new card	Checks form processing
3	Submit same register/date/session again	Duplicate warning	Checks duplicate rule
4	Enter a register number in Search	Matching entry appears	Checks keyword search
5	Choose Present filter	Only Present records appear	Checks status filter
6	Choose Absent filter	Only Absent records appear	Checks status filter
7	Refresh the browser page	Saved records remain	Checks localStorage
8	Search for unmatched text	No matching records message	Checks empty results
9	Use Clear Demo Data and confirm	Records are cleared	Checks clear control
10	Cancel clear confirmation	Records remain unchanged	Checks confirmation flow

Acceptance criteria

The prototype can be considered to meet its basic objective when a user can enter a valid attendance record, see it in the records section, locate it through search, filter by attendance status, and see the dashboard counts reflect the stored records. Duplicate combinations should be rejected with feedback.

Technical Concepts — Expanded Notes

1. Separation of concerns

HTML, CSS and JavaScript have different responsibilities. HTML provides the structure and meaningful controls. CSS determines presentation and responsive layout. JavaScript handles data and interaction. Keeping these responsibilities separate makes it easier to update the page design without rewriting the storage logic.

2. Arrays and objects

The records variable is an array. Each item in that array is an object containing properties such as name, registerNo, department, date, status, session and notes. An array is suitable for a list of records, while an object keeps related details together.

3. Array processing

The some() method checks whether any existing record matches the duplicate rule. The filter() method creates a subset of records that match the search text and selected status. The map() method converts each matching record into a card representation. These methods make the code shorter and easier to read than repeating manual loops for each task.

4. Browser storage

localStorage persists string values between page visits in the same browser profile. Since the application stores an array of objects, JSON.stringify() converts the array to text before saving. JSON.parse() converts the stored text back to JavaScript data when the page opens. If browser storage is cleared, the saved entries can be lost.

5. DOM updates

The Document Object Model (DOM) represents the loaded page as elements that JavaScript can access. getElementById() finds an element using its ID. textContent changes the visible text in dashboard counters. innerHTML is used to display the record cards after user values are escaped. Event listeners respond to form submission and keep the interface updated.

6. User feedback

A short toast message informs the user about a successful save, a duplicate entry or a clear action. Feedback is important because it tells the user whether the requested operation was completed. The duplicate message also explains why the entry was not added.